



Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

SQL with sqlite3

WEEK 3 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Week 3 opens: the book in a database

Courses 1 and 2 answered every question about the book with pandas. On many desks the same data sits in a database, and SQL is how you ask it. Today the synthetic book goes into one database file.

Every query sits beside its pandas version, and an assert proves the two return the same rows.

No live cells today: everything comes from the course's files, so no FRED key is needed.

Week 3: SQL, AI, and the snapshot

- | | |
|--------------|------------------------------|
| Day 1 | SQL with sqlite3 |
| Day 2 | SQL joins, time series |
| Day 3 | Working with an AI assistant |
| Day 4 | The daily market snapshot |
| Day 5 | The capstone |

Why SQL: you already know most of it

Job	pandas	SQL
Pick columns	<code>holdings[["cusip", "oas"]]</code>	<code>SELECT cusip, oas FROM holdings</code>
Pick rows	<code>holdings[holdings["oas"] > 200]</code>	<code>WHERE oas > 200</code>
Sort, first five	<code>.sort_values(...).head(5)</code>	<code>ORDER BY oas DESC LIMIT 5</code>
Group and total	<code>.groupby("sector").agg(...)</code>	<code>GROUP BY sector, COUNT, SUM, AVG</code>
Keep some groups	a filter after groupby	<code>HAVING</code>
Join	<code>pd.merge(...)</code>	<code>JOIN (tomorrow)</code>

Assistants write SQL readily, and it runs whether it is right or not. Read it, then check it against pandas.

sqlite3: a database in one file

One file

The whole database is a file the program reads and writes. No server, no password.

Built in

Python ships the sqlite3 module: SQLite 3.50.4 here.

Everywhere

"Likely used more than all other database engines combined."

```
connection.execute("SELECT 7 / 2, 7 / 2.0")  
7 / 2 = 3, 7 / 2.0 = 3.5
```

Two whole numbers divide as whole numbers. Write * 1.0 on a count.

SQLite, "Single-file Cross-platform Database" and "Most Widely Deployed and Used Database Engine", sqlite.org (read 2026-10-04).

load_tables: CSV in, counts checked

```
frame = pd.read_csv(csv_path)
frame.to_sql(name, connection, if_exists="replace")
# read the count back and check it
SELECT COUNT(*) FROM {name}
```

A table name cannot be a ? parameter, so only plain names (letters, digits, underscore) are allowed.

```
b'SQLite format 3\x00'
```

Seven tables, each count checked

holdings	200 rows
issuers	150 rows
ratings	18 rows
benchmark	821 rows
trades	374 rows
treasury	35,728 rows
spreads	2,940 rows

A database file is binary and rebuilt in one call: *.db stays in .gitignore.

SELECT, WHERE, ORDER BY, LIMIT

```
SELECT cusip, issuer, market_value  
FROM holdings  
WHERE sector = ? AND oas > ?  
ORDER BY market_value DESC  
LIMIT ?
```

run_query

One query, a DataFrame back; values through params; the connection closed.

assert_same_rows

Sorted by your keys, index reset; text exact, decimals to rtol 1e-12.

Without ORDER BY, a database promises no order: LIMIT 5 alone is five rows, not the first five.

Dates are ISO text, YYYY-MM-DD: year first, so < on text compares in date order. They come back as text; convert with `pd.to_datetime`.

Values go in as ? parameters

Pasted into the SQL with an f-string, a value becomes SQL: a quote can break the query or change it. A ? parameter is sent separately and is only ever a value.

A ticker with a quote, pasted: O'BRN

```
DatabaseError: ... near "BRN": syntax error
```

A value that rewrites the question: NONE' OR '1'='1

200 rows

pasted into the SQL: every bond, and no error

0 rows

as a ? parameter: a strange ticker, no match

This is SQL injection. Only a checked table or column name ever goes into the text.

GROUP BY, then HAVING

```
SELECT sector, COUNT(*) AS bonds
FROM holdings
WHERE oas > ?          -- rows first
GROUP BY sector
HAVING COUNT(*) >= ?   -- then groups
ORDER BY sector
```

The clauses, always in this order

SELECT	FROM
WHERE	GROUP BY
HAVING	ORDER BY
LIMIT	

WHERE filters rows before grouping. HAVING filters groups after. A single row has no COUNT.

HAVING COUNT(*) >= 20 keeps 7 of 10 sectors. 46 tests pass at the end of the section.

A market-value weighted average in SQL

```
SUM(oas * market_value) / SUM(market_value) AS weighted_oas
```

AVG(oas) gives every bond the same say. The weighted average gives every dollar the same say, and bondmath.weighted_average is the answer key.

180.46 bp

the book's market-value weighted OAS, 200 bonds (synthetic)

185.75 bp

the simple AVG(oas) of the same bonds

A query that writes AVG where the question said weighted runs, looks plausible, and is wrong.

Excel side by side

SQL	Excel (tested; long ranges shortened)	Energy
COUNT(*) ... GROUP BY	=COUNTIFS(\$C\$2:\$C\$201, J2)	17
SUM(market_value)	=SUMIFS(\$F\$2:\$F\$201, \$C\$2:\$C\$201, J2)	38,830,005
AVG(oas)	=AVERAGEIFS(\$E\$2:\$E\$201, \$C\$2:\$C\$201, J2)	171.5882
WHERE ... AND oas > 200	=COUNTIFS(. . . , J2, \$E\$2:\$E\$201, ">200")	3
weighted OAS	=SUMPRODUCT((C=J2)*E*F)/SUMIFS(F, C, J2)	176.3494

WHERE is FILTER; ORDER BY ... DESC LIMIT 5 is TAKE(SORTBY(...,-1),5); HAVING is FILTER on the summary block.

A pivot table does the whole GROUP BY at once: 10 sectors, Grand Total 200 bonds.

Every formula typed into Excel 16.0 build 20430 by a script on 2026-10-04, equal to the SQL and pandas results.

Save the day: the database stays out

```
git check-ignore -v course3.db
.gitignore:5:*.db      course3.db
git status --short --ignored
!! course3.db
```

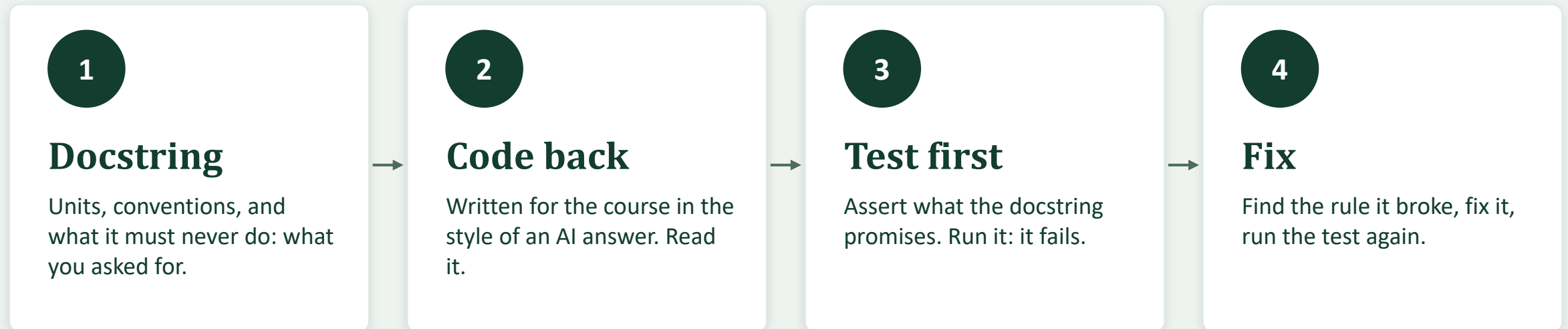
Committed: four text files

```
.gitignore
marketdata.py
test_marketdata.py
queries.sql
```

Binary, rewritten on every load, rebuilt from the CSVs in one call: the CSVs and the code are the record.

The AI check

The docstring is the prompt. The code that comes back runs, and has one data bug.



Today: "weighted OAS by grade" written as `AVG(oas)`. It runs and returns plausible spreads; bondmath disagrees.

Code that runs is not the same as code that is right.

What you can do now

- A SQLite database in one file, kept out of Git with *.db
- load_tables: CSVs to tables, every row count checked
- SELECT, WHERE, ORDER BY, LIMIT, each proved against pandas
- Every value as a ? parameter, never an f-string
- GROUP BY with HAVING; a weighted OAS that bondmath agrees with

Week 3, Day 2 SQL joins and time series: matching tables, the duplicating join, and dates in SQL